

vLLM专题（十一）-工具调用(Tool Calling)

原创

AI专题精讲

已于 2025-02-24 21:09:22 修改

阅读量891

★ 收藏

👍 点赞数

CC 4.0 BY-SA版权

分类专栏：

大模型专题系列

 文章标签：

人工智能

 大模型专题系列 专栏收录该内容

👑 您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益 >

vLLM 目前支持命名函数调用，以及在聊天补全 API 的 `tool_choice` 字段中支持 `auto` 和 `none` 选项。`tool_choice` 选项中的 `required` 尚未支持，但已在计划中。

1. 快速开始

启动服务器并启用工具调用功能。此示例使用 Meta 的 Llama 3.1 8B 模型，因此我们需要使用 vLLM 示例目录中的 llama3 工具调用聊天模板：

```
vllm serve meta-llama/Llama-3.1-8B-Instruct \
--enable-auto-tool-choice \
--tool-call-parser llama3_json \
--chat-template examples/tool_chat_template_llama3.1_json.jinja
```

接下来，向模型发起一个请求，使其使用可用的工具：

```
from openai import OpenAI
import json

client = OpenAI(base_url="http://localhost:8000/v1", api_key="dummy")

def get_weather(location: str, unit: str):
    return f"Getting the weather for {
        location} in {
        unit}..."
tool_functions = {
    "get_weather": get_weather}

tools = [{

    "type": "function",
    "function": {

        "name": "get_weather",
```

¥49.90
¥99.00

111 篇文章

已订阅

8折续

```

        "description": "Get the current weather in a given location",
        "parameters": {

            "type": "object",
            "properties": {

                "location": {
                    "type": "string", "description": "City and state, e.g., 'San Francisco, CA'"
                },

                "unit": {
                    "type": "string", "enum": ["celsius", "fahrenheit"]}
            },
            "required": ["location", "unit"]
        }
    }
}]]

response = client.chat.completions.create(
    model=client.models.list().data[0].id,
    messages=[{
        "role": "user", "content": "What's the weather like in San Francisco?"}],
    tools=tools,
    tool_choice="auto"
)

tool_call = response.choices[0].message.tool_calls[0].function
print(f"Function called: {
    tool_call.name}")
print(f"Arguments: {
    tool_call.arguments}")
print(f"Result: {
    get_weather(**json.loads(tool_call.arguments))}")

```

示例输出：

```

Function called: get_weather
Arguments: {
    "location": "San Francisco, CA", "unit": "fahrenheit"}
Result: Getting the weather for San Francisco, CA in fahrenheit...

```

这个示例演示了：

1. 设置启用工具调用的服务器
2. 定义一个实际的函数来处理工具调用
3. 使用 `tool_choice="auto"` 发起请求
4. 处理结构化响应并执行相应的函数

你也可以通过设置 `tool_choice={"type": "function", "function": {"name": "get_weather"}}` 来指定一个特定的函数。请注意，这将使用引导解码后端——所以第一次使用时，会有几秒钟的延迟（甚至更长），因为有限状态机（FSM）需要首次编译，然后缓存以供后续请求使用。

请记住，调用者有责任：

1. 在请求中定义适当的工具
2. 在聊天消息中包括相关上下文
3. 在应用程序逻辑中处理工具调用

有关更高级的用法，包括并行工具调用和不同模型特定的解析器，请参阅下面的章节。

2. 命名函数调用

vLLM 默认支持在聊天补全 API 中使用命名函数调用。这是通过 Outlines 的引导解码实现的，因此默认启用，并且适用于所有支持的模型。你可以确保获得一个可解析的函数调用——尽管不一定是高质量的。

vLLM 将使用引导解码来确保响应与 `tools` 参数中定义的 JSON 模式工具参数对象匹配。为了获得最佳结果，我们建议在提示中明确指定预期的输出格式或模式，以确保模型的生成意图与引导解码后端强制生成的模式保持一致。

要使用命名函数，你需要在聊天补全请求的 `tools` 参数中定义函数，并在 `tool_choice` 参数中指定其中一个工具的名称。

3. 自动函数调用

要启用此功能，你需要设置以下标志：

`--enable-auto-tool-choice` – **必选**。自动工具选择。告诉 vLLM 你希望启用模型在认为合适时自动生成工具调用。

`--tool-call-parser` – 选择要使用的工具解析器（如下所列）。未来将继续添加更多工具解析器，你也可以通过 `--tool-parser-plugin` 注册自定义的工具解析器。

`--tool-parser-plugin` – **可选**。工具解析器插件，用于将用户定义的工具解析器注册到 vLLM 中，注册的工具解析器名称可以在 `--tool-call-parser` 中指定。

`--chat-template` – **可选**，用于自动工具选择。指定处理工具角色消息和包含先前生成工具调用的助手角色消息的聊天模板路径。Hermes、Mistral 和 Llama 模型的 `tokenizer_config.json` 文件中已经包含了兼容工具的聊天模板，但你可以指定自定义模板。如果你的模型在 `tokenizer_config.json` 中配置了特定于工具使用的聊天模板，可以将此参数设置为 `tool_use`。在这种情况下，将根据 transformers 规范使用该模板。更多信息可以参考 HuggingFace 的说明；你可以在 [这里](#) 找到一个 `tokenizer_config.json` 的示例。

如果你喜欢的工具调用模型尚未支持，欢迎贡献一个解析器和工具使用的聊天模板！

3.1 Hermes 模型 (hermes)

所有比 Hermes 2 Pro 更新的 Nous Research Hermes 系列模型都应支持。

- `NousResearch/Hermes-2-Pro-*`
- `NousResearch/Hermes-2-Theta-*`
- `NousResearch/Hermes-3-*`

需要注意的是，Hermes 2 Theta 模型由于创建过程中的合并步骤，已知其工具调用质量和能力有所下降。

标志: `--tool-call-parser hermes`

3.2 Mistral 模型 (mistral)

支持的模型:

- `mistralai/Mistral-7B-Instruct-v0.3`（已确认）

其他 Mistral 函数调用模型也兼容。

已知问题:

Mistral 7B 在生成并行工具调用时表现不佳。

Mistral 的 `tokenizer_config.json` 聊天模板要求工具调用 ID 必须为 9 位数字，这比 vLLM 生成的 ID 短得多。由于不满足此条件时会抛出异常，因此提供了以下额外的聊天模板：

- `examples/tool_chat_template_mistral.jinja` - 这是“官方”Mistral 聊天模板，但经过调整以适配 vLLM 的工具调用 ID（提供的 `tool_call_id` 字段被截断为最后 9 位数字）。
- `examples/tool_chat_template_mistral_parallel.jinja` - 这是一个“更好”的版本，当提供工具时，会添加一个工具使用系统提示，从而在并行工具调用时显著提高可靠性。

推荐标志：

```
--tool-call-parser mistral --chat-template
examples/tool_chat_template_mistral_parallel.jinja
```

3.3 Llama 模型 (llama3_json)

支持的模型：

- `meta-llama/Meta-Llama-3.1-8B-Instruct`
- `meta-llama/Meta-Llama-3.1-70B-Instruct`
- `meta-llama/Meta-Llama-3.1-405B-Instruct`
- `meta-llama/Meta-Llama-3.1-405B-Instruct-FP8`

支持的工具调用是基于 JSON 的工具调用。对于 Llama-3.2 模型中的 Python 风格工具调用，请参阅下面的 Python 风格工具解析器。不支持其他工具调用格式，例如内置的 Python 工具调用或自定义工具调用。

已知问题：

- 不支持并行工具调用。
- 模型可能会生成格式错误的参数，例如将数组序列化为字符串而不是数组。

`tool_chat_template_llama3_json.jinja` 文件包含“官方”Llama 聊天模板，但经过调整以更好地适配 vLLM。

推荐标志：

```
--tool-call-parser llama3_json --chat-template
examples/tool_chat_template_llama3_json.jinja
```

3.4 IBM Granite

支持的模型：

`ibm-granite/granite-3.0-8b-instruct`

- 推荐标志：`--tool-call-parser granite --chat-template examples/tool_chat_template_granite.jinja`
- `examples/tool_chat_template_granite.jinja`：这是基于 Huggingface 原始模板修改的聊天模板，支持并行函数调用。

`ibm-granite/granite-3.1-8b-instruct`

- 推荐标志：`--tool-call-parser granite`
- 可以直接使用 Huggingface 的聊天模板，支持并行函数调用。

`ibm-granite/granite-20b-functioncalling`

- 推荐标志：`--tool-call-parser granite-20b-fc --chat-template examples/tool_chat_template_granite_20b_fc.jinja`
- `examples/tool_chat_template_granite_20b_fc.jinja`：这是基于 Huggingface 原始模板修改的聊天模板，原始模板不兼容 vLLM。它融合了 Hermes 模板的函数描述元素，并遵循论文中“响应生成”模式的系统提示，支持并行函数调用。

3.5 InternLM 模型 (internlm)

支持的模型：

- `internlm/internlm2_5-7b-chat`（已确认）

其他 `internlm2.5` 函数调用模型也兼容。

已知问题：

- 尽管此实现也支持 InternLM2，但在使用 `internlm/internlm2-chat-7b` 模型测试时，工具调用结果不稳定。

推荐标志：

```
--tool-call-parser internlm --chat-template  
examples/tool_chat_template_internlm2_tool.jinja
```

3.6 Jamba 模型 (jamba)

AI21 的 Jamba-1.5 模型已支持。

- [ai21labs/AI21-Jamba-1.5-Mini](#)
- [ai21labs/AI21-Jamba-1.5-Large](#)

标志: `--tool-call-parser jamba`

3.7 支持 Python 风格工具调用的模型 (pythonic)

越来越多的模型使用 Python 列表来表示工具调用，而不是 JSON。这种方式的优势在于天然支持并行工具调用，并消除了工具调用所需的 JSON 模式的歧义。Python 风格工具解析器可以支持此类模型。

作为一个具体示例，这些模型可能会通过生成以下内容来查询旧金山和西雅图的天气：

```
[get_weather(city='San Francisco', metric='celsius'), get_weather(city='Seattle', metric='celsius')]
```

限制：

- 模型不能在同一生成过程中同时生成文本和工具调用。对于特定模型来说，这可能不难修改，但社区目前对于开始和结束工具调用时应发出哪些标记尚未达成共识。（特别是 Llama 3.2 模型不会发出此类标记。）
- Llama 的小模型在使用工具时表现不佳。

示例支持的模型：

- [meta-llama/Llama-3.2-1B-Instruct*](#) （使用 `examples/tool_chat_template_llama3.2_pythonic.jinja` ）
- [meta-llama/Llama-3.2-3B-Instruct*](#) （使用 `examples/tool_chat_template_llama3.2_pythonic.jinja` ）
- [Team-ACE/ToolACE-8B](#) （使用 `examples/tool_chat_template_toolace.jinja` ）
- [fixie-ai/ultravox-v0_4-ToolACE-8B](#) （使用 `examples/tool_chat_template_toolace.jinja` ）

标志：

`--tool-call-parser pythonic --chat-template {见上文}`

警告：Llama 的小模型经常无法以正确格式发出工具调用。实际效果可能因模型而异。

4. 如何编写工具解析器插件

工具解析器插件是一个包含一个或多个 `ToolParser` 实现的 Python 文件。你可以参考 [vllm/entrypoints/openai/tool_parsers/hermes_tool_parser.py](#) 中的 `Hermes2ProToolParser` 来编写 `ToolParser` 。

以下是插件文件的概要：

```
# import the required packages

# define a tool parser and register it to vllm
# the name list in register_module can be used
# in --tool-call-parser. you can define as many
# tool parsers as you want here.
@ToolParserManager.register_module(["example"])
class ExampleToolParser(ToolParser):
    def __init__(self, tokenizer: AnyTokenizer):
        super().__init__(tokenizer)

    # adjust request. e.g.: set skip special tokens
    # to False for tool call output.
    def adjust_request(
        self, request: ChatCompletionRequest) -> ChatCompletionRequest:
        return request

    # implement the tool call parse for stream call
    def extract_tool_calls_streaming(
        self,
        previous_text: str,
        current_text: str,
        delta_text: str,
        previous_token_ids: Sequence[int],
        current_token_ids: Sequence[int],
        delta_token_ids: Sequence[int],
        request: ChatCompletionRequest,
    ) -> Union[DeltaMessage, None]:
        return delta

    # implement the tool parse for non-stream call
    def extract_tool_calls(
        self,
        model_output: str,
        request: ChatCompletionRequest,
    ) -> ExtractedToolCallInformation:
        return ExtractedToolCallInformation(tools_called=False,
                                             tool_calls=[],
                                             content=text)
```

然后，你可以在命令行中像这样使用此插件：


```
--enable-auto-tool-choice \  
--tool-parser-plugin <absolute path of the plugin file>  
--tool-call-parser example \  
--chat-template <your chat template> \  
``
```



AI专题精讲

已关注



分享 ...

专栏目录

已订阅